

Chapter 31 — Greedy

0. What You Will Learn in This Chapter

Greedy is one of the most important and also one of the most dangerous interview patterns.

A greedy algorithm makes the **best local choice now** and hopes that repeated local choices produce a globally optimal answer.

Greedy problems often look simple after you know the trick, but they are difficult because you must justify:

Why is this local choice always safe?

By the end of this chapter, you should understand:

- What greedy means
- When greedy works
- When greedy fails
- How to identify greedy problems
- How to prove greedy correctness
- Sorting + greedy
- Interval greedy
- Two-pointer greedy
- Heap + greedy
- Stack + greedy
- Greedy with counting
- Greedy with prefix/suffix thinking
- Greedy for jump problems
- Greedy for scheduling problems
- Greedy for resource allocation
- Greedy for hard interview problems

Core problems covered in this chapter:

1. Assign Cookies
 2. Lemonade Change
 3. Jump Game
 4. Jump Game II
 5. Gas Station
 6. Partition Labels
 7. Boats to Save People
 8. Non-overlapping Intervals
 9. Minimum Number of Arrows to Burst Balloons
 10. Queue Reconstruction by Height
 11. Candy
 12. Task Scheduler
 13. Hand of Straights
 14. Car Fleet
 15. Patching Array
-

1. What Is Greedy?

A greedy algorithm makes the best possible choice at the current moment. It does **not** try all possibilities.

It does **not** go back and change earlier decisions.

It commits to a local choice.

Example:

Suppose you want to satisfy children with cookies.

Each child has greed factor.

Each cookie has size.

A child is satisfied if:

cookie size $\geq$ greed factor

Greedy idea:

Give the smallest cookie that can satisfy the least greedy child.

Why?

Because wasting a large cookie on a less greedy child may prevent satisfying a more greedy child later.

2. Why Greedy Exists

Many brute-force problems try all possible choices.

Example:

Assign cookies to children.

Brute force:

Try every cookie-child assignment.

This is exponential.

Greedy avoids this by proving that one local choice is always safe.

For Assign Cookies:

Smallest sufficient cookie should go to the least greedy remaining child.

Once that choice is made, we never need to reconsider it.

3. Core Greedy Requirement

Greedy works only when the problem has a **safe local choice**.

A safe choice means:

There exists an optimal solution that includes this choice.

If we can prove that, then we can make the greedy choice and reduce the problem.

4. Greedy vs Dynamic Programming

Greedy and DP both solve optimization problems, but they are different.

Decision style	take best local choice	try multiple choices
Revisits choices?	no	yes
Needs proof?	yes, strongly	recurrence proof

Feature	Greedy	Dynamic Programming
Typical time	often $O(n)$ or $O(n \log n)$	often $O(n)$, $O(n^2)$, or more
Example	Jump Game	Coin Change
Risk	local choice may fail	state may be hard to define

Example where greedy fails:

Coin Change with coins:

coins = [1, 3, 4]

amount = 6

Greedy chooses:

$4 + 1 + 1 = 3$ coins

Optimal is:

$3 + 3 = 2$ coins

So this needs DP, not greedy.

5. When to Use Greedy

Use greedy when the problem asks for:

5.1 Minimum or Maximum With Local Choice

Examples:

- minimum jumps
- maximum satisfied children
- minimum arrows
- minimum boats
- minimum removals

5.2 Sorting Helps Choose Best Order

Examples:

- Assign Cookies
- Boats to Save People
- Non-overlapping Intervals
- Queue Reconstruction

5.3 Intervals and Scheduling

Examples:

- Non-overlapping Intervals
- Minimum Arrows
- Meeting Rooms
- Activity Selection

5.4 Resource Allocation

Examples:

- Lemonade Change
- Task Scheduler
- Furthest Building
- IPO

5.5 Reachability / Farthest Coverage

Examples:

- Jump Game
- Jump Game II
- Patching Array

5.6 Greedy Stack Problems

Examples:

- Remove K Digits
 - Remove Duplicate Letters
 - Create Maximum Number
-

6. When NOT to Use Greedy

Do not use greedy when:

6.1 Local Best Choice Can Block Global Best

Example:

Coin Change with [1,3,4].

6.2 Problem Requires Trying All Choices

Use backtracking.

Examples:

- N-Queens
- Word Search
- Subsets
- Permutations

6.3 Problem Has Overlapping Subproblems

Use DP.

Examples:

- House Robber
- Longest Common Subsequence
- Edit Distance

6.4 There Is No Clear Proof

If you cannot explain why your greedy choice is safe, be suspicious.

7. Mental Models

7.1 Stay Ahead

A greedy solution is correct if after every step it is at least as good as any other solution.

Example:

Jump Game keeps the farthest reachable index.

If greedy farthest cannot reach the end, no other method can.

7.2 Earliest Finish Time

For interval scheduling, always choose the interval that ends earliest.

Why?

It leaves the most room for future intervals.

7.3 Use Smallest Sufficient Resource

Give the smallest resource that satisfies the current need.

Examples:

- Assign Cookies
- Boats to Save People

7.4 Spend Scarce Resource Wisely

Use scarce resources only when necessary.

Example:

Lemonade Change:

Prefer giving one \$10 and one \$5 as change for \$20, instead of three \$5 bills, because \$5 bills are more flexible.

7.5 Cover the Largest Range

Some greedy problems maintain a covered range.

Example:

Patching Array tracks:

we can currently form every sum in [1, miss)

If next number is too large, patch with miss.

8. Pattern Recognition Checklist

Ask:

16. Does the problem ask for minimum or maximum?
17. Can sorting make local choices easier?
18. Is there a natural “best next choice”?
19. Can I prove that this choice never hurts?
20. Does the problem involve intervals?
21. Does the problem involve resources?
22. Does the problem involve reachability or farthest coverage?
23. Does the problem involve scheduling?
24. Would DP be overkill if a local proof exists?
25. Can I explain the greedy proof in one paragraph?

If yes, greedy may apply.

9. Common Greedy Proof Techniques

9.1 Exchange Argument

Show that if an optimal solution does not use your greedy choice, you can swap something with your greedy choice and remain optimal.

Used in:

- Assign Cookies
- Non-overlapping Intervals
- Minimum Arrows

9.2 Staying Ahead

Show greedy is always at least as far ahead as any alternative.

Used in:

- Jump Game
- Jump Game II
- Patching Array

9.3 Cut / Boundary Argument

Show that after a boundary is fixed, the greedy decision is forced.

Used in:

- Partition Labels
- Gas Station

9.4 Contradiction

Assume greedy choice is wrong, then show contradiction with optimality.

10. Core Greedy Templates

Template 1 — Sort + Greedy Scan

```
sort(nums.begin(), nums.end());

for (int x : nums) {
    // make local best choice
}
```

Used for:

- Assign Cookies
 - Maximum Units on a Truck
 - Boats to Save People
-

Template 2 — Interval Greedy by End Time

```
sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const vector<int>& b) {
    return a[1] < b[1];
});

int count = 0;
int lastEnd = INT_MIN;

for (auto& interval : intervals) {
    if (interval[0] >= lastEnd) {
        count++;
        lastEnd = interval[1];
    }
}
```

Used for:

- Non-overlapping Intervals
 - Activity Selection
-

Template 3 — Farthest Reach

```
int farthest = 0;

for (int i = 0; i < nums.size(); i++) {
    if (i > farthest) {
        return false;
    }

    farthest = max(farthest, i + nums[i]);
}

return true;
```

Used for:

- Jump Game
-

Template 4 — Greedy Level Expansion

```
int jumps = 0;
int currentEnd = 0;
int farthest = 0;

for (int i = 0; i < nums.size() - 1; i++) {
    farthest = max(farthest, i + nums[i]);

    if (i == currentEnd) {
        jumps++;
        currentEnd = farthest;
    }
}

return jumps;
```

Used for:

- Jump Game II
-

Template 5 — Greedy With Counts

```
unordered_map<int, int> count;

for (int x : nums) {
    count[x]++;
}

for (int x : sortedNums) {
```

```
// consume counts greedily
}
```

Used for:

- Hand of Straights
- Divide Array into Consecutive Sets

11. Major Greedy Variations

Sort + assign	smallest sufficient resource	Assign Cookies
Money/change	preserve flexible bills	Lemonade Change
Farthest reach	keep max reachable index	Jump Game
Layer greedy	finish current range then jump	Jump Game II
Reset point	bad prefix cannot be start	Gas Station
Partition boundary	close segment at last occurrence	Partition Labels
Two-pointer greedy	pair extremes	Boats to Save People
Interval end greedy	earliest finish leaves room	Non-overlapping Intervals
Coverage greedy	one action covers max range	Minimum Arrows
Heap greedy	choose current most urgent item	Task Scheduler
Counting greedy	build required groups	Hand of Straights
Patch coverage	extend constructible range	Patching Array

12. Full Problem Walkthrough 1 — LeetCode 455: Assign Cookies

12.1 Problem Summary

Each child has greed factor $g[i]$.

Each cookie has size $s[j]$.

A child is content if:

$s[j] \geq g[i]$

Return maximum number of content children.

12.2 How to Identify Greedy

We need maximize satisfied children.

A small cookie should be used on the least greedy child it can satisfy.

Sort both arrays.

12.3 C++ Code

```
class Solution {
public:
    int findContentChildren(vector<int>& g, vector<int>& s) {
        sort(g.begin(), g.end());
        sort(s.begin(), s.end());

        int child = 0;
        int cookie = 0;

        while (child < g.size() && cookie < s.size()) {
            if (s[cookie] >= g[child]) {
                child++;
            }

            cookie++;
        }

        return child;
    }
};
```

12.4 Greedy Proof

Always satisfy the least greedy remaining child using the smallest cookie that works.

If the smallest cookie cannot satisfy the least greedy child, it cannot satisfy any other remaining child either, so skipping it is safe.

12.5 Complexity

Time:

$O(n \log n + m \log m)$

Space:

$O(1)$ extra

13. Full Problem Walkthrough 2 — LeetCode 860: Lemonade Change

13.1 Problem Summary

Each lemonade costs \$5.

Customers pay with:

\$5, \$10, or \$20

Return true if you can give correct change to every customer.

13.2 How to Identify Greedy

Need preserve smaller bills for future change.

When customer pays \$20, prefer giving:

\$10 + \$5

instead of:

\$5 + \$5 + \$5

because \$5 bills are more flexible.

13.3 C++ Code

```
class Solution {
public:
    bool lemonadeChange(vector<int>& bills) {
        int five = 0;
        int ten = 0;

        for (int bill : bills) {
            if (bill == 5) {
                five++;
            } else if (bill == 10) {
                if (five == 0) {
                    return false;
                }

                five--;
                ten++;
            } else {
                if (ten > 0 && five > 0) {
                    ten--;
                    five--;
                } else if (five >= 3) {
                    five -= 3;
                } else {
                    return false;
                }
            }
        }

        return true;
    }
};
```

```
}  
};
```

13.4 Complexity

Time:

$O(n)$

Space:

$O(1)$

14. Full Problem Walkthrough 3 — LeetCode 55: Jump Game

14.1 Problem Summary

Given `nums[i]`, maximum jump length from index `i`.

Return true if you can reach last index.

14.2 How to Identify Greedy

Need reachability.

At every index, maintain farthest reachable position.

If current index is beyond farthest, impossible.

14.3 C++ Code

```
class Solution {  
public:  
    bool canJump(vector<int>& nums) {  
        int farthest = 0;  
  
        for (int i = 0; i < nums.size(); i++) {  
            if (i > farthest) {  
                return false;  
            }  
  
            farthest = max(farthest, i + nums[i]);  
        }  
  
        return true;  
    }  
};
```

14.4 Greedy Proof

If index `i` is reachable, then all positions up to the farthest reachable index are also candidates.

Keeping the maximum farthest reach is always at least as good as choosing any specific jump early.

14.5 Complexity

Time:

$O(n)$

Space:

$O(1)$

15. Full Problem Walkthrough 4 — LeetCode 45: Jump Game II

15.1 Problem Summary

Return minimum number of jumps to reach last index.

15.2 How to Identify Greedy

This is like BFS levels without using a queue.

Current jump covers range:

[previousEnd + 1 ... currentEnd]

Within this range, find farthest next reach.

When current index reaches currentEnd, we must make another jump.

15.3 C++ Code

```
class Solution {
public:
    int jump(vector<int>& nums) {
        int jumps = 0;
        int currentEnd = 0;
        int farthest = 0;

        for (int i = 0; i < nums.size() - 1; i++) {
            farthest = max(farthest, i + nums[i]);

            if (i == currentEnd) {
                jumps++;
                currentEnd = farthest;
            }
        }

        return jumps;
    }
};
```

15.4 Greedy Proof

Each jump should extend reach as far as possible among all positions reachable with the current number of jumps.
This mimics BFS levels, so the first time we cover the end uses the minimum number of jumps.

15.5 Complexity

Time:

$O(n)$

Space:

$O(1)$

16. Full Problem Walkthrough 5 — LeetCode 134: Gas Station

16.1 Problem Summary

There are gas stations in a circle.

At station i :

- gain $\text{gas}[i]$
- spend $\text{cost}[i]$ to go next

Return starting station index if possible, else -1 .

16.2 How to Identify Greedy

If total gas is less than total cost, impossible.

If starting at start fails at station i , then no station between start and i can be a valid start.

So reset start to $i + 1$.

16.3 C++ Code

```
class Solution {
public:
    int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {
        int total = 0;
        int tank = 0;
        int start = 0;

        for (int i = 0; i < gas.size(); i++) {
            int diff = gas[i] - cost[i];

            total += diff;
            tank += diff;

            if (tank < 0) {
                start = i + 1;
                tank = 0;
            }
        }

        return total < 0 ? -1 : start;
    }
};
```

```

    }
}

return total >= 0 ? start : -1;
}
};

```

16.4 Greedy Proof

If we cannot reach station $i + 1$ from current start, then every station after start up to i also fails, because they start with even less accumulated surplus.

Therefore, the next possible candidate is $i + 1$.

16.5 Complexity

Time:

$O(n)$

Space:

$O(1)$

17. Full Problem Walkthrough 6 — LeetCode 763: Partition Labels

17.1 Problem Summary

Partition string so each letter appears in at most one part.

Return sizes of parts.

Example:

```
s = "ababcbacadefegdehijhklij"
```

Output:

```
[9,7,8]
```

17.2 How to Identify Greedy

Each character has a last occurrence.

A partition must extend at least to the last occurrence of every character inside it.

When current index reaches the farthest last occurrence, close the partition.

17.3 C++ Code

```

class Solution {
public:
    vector<int> partitionLabels(string s) {
        vector<int> last(26, 0);

        for (int i = 0; i < s.size(); i++) {
            last[s[i] - 'a'] = i;
        }
    }
}

```

```

vector<int> result;

int start = 0;
int end = 0;

for (int i = 0; i < s.size(); i++) {
    end = max(end, last[s[i] - 'a']);

    if (i == end) {
        result.push_back(end - start + 1);
        start = i + 1;
    }
}

return result;
}
};

```

17.4 Greedy Proof

A partition cannot end before the last occurrence of any character inside it. Once current index reaches the farthest required boundary, ending there is safe and gives the smallest possible current partition.

17.5 Complexity

Time:

$O(n)$

Space:

$O(1)$

18. Full Problem Walkthrough 7 — LeetCode 881: Boats to Save People

18.1 Problem Summary

Each boat carries at most two people with total weight at most limit. Return minimum number of boats.

18.2 How to Identify Greedy

Sort weights.

The heaviest remaining person must take a boat.

Try pairing them with the lightest remaining person.

18.3 C++ Code

```
class Solution {
public:
    int numRescueBoats(vector<int>& people, int limit) {
        sort(people.begin(), people.end());

        int left = 0;
        int right = people.size() - 1;
        int boats = 0;

        while (left <= right) {
            if (people[left] + people[right] <= limit) {
                left++;
            }

            right--;
            boats++;
        }

        return boats;
    }
};
```

18.4 Greedy Proof

The heaviest person must be placed in some boat.

If the lightest person cannot fit with the heaviest, nobody else can.

If the lightest can fit, pairing them is safe because it uses the least possible extra weight.

18.5 Complexity

Time:

$O(n \log n)$

Space:

$O(1)$ extra

19. Full Problem Walkthrough 8 — LeetCode 435: Non-overlapping Intervals

19.1 Problem Summary

Given intervals, return minimum number of intervals to remove so remaining intervals do not overlap.

19.2 How to Identify Greedy

Equivalent:

Keep maximum number of non-overlapping intervals.

Choose intervals ending earliest.

19.3 C++ Code

```
class Solution {
public:
    int eraseOverlapIntervals(vector<vector<int>>& intervals) {
        if (intervals.empty()) {
            return 0;
        }

        sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const
vector<int>& b) {
            return a[1] < b[1];
        });

        int removals = 0;
        int lastEnd = intervals[0][1];

        for (int i = 1; i < intervals.size(); i++) {
            if (intervals[i][0] < lastEnd) {
                removals++;
            } else {
                lastEnd = intervals[i][1];
            }
        }

        return removals;
    }
};
```

19.4 Greedy Proof

Choosing the interval that ends earliest leaves maximum room for future intervals.

Any overlapping interval ending later is never better to keep.

19.5 Complexity

Time:

$O(n \log n)$

Space:

$O(1)$ extra

20. Full Problem Walkthrough 9 — LeetCode 452: Minimum Number of Arrows to Burst Balloons

20.1 Problem Summary

Each balloon is an interval.

One arrow at position x bursts all balloons containing x .

Return minimum arrows.

20.2 How to Identify Greedy

Sort balloons by end.

Shoot arrow at end of first balloon.

This covers as many overlapping balloons as possible.

20.3 C++ Code

```
class Solution {
public:
    int findMinArrowShots(vector<vector<int>>& points) {
        if (points.empty()) {
            return 0;
        }

        sort(points.begin(), points.end(), [](const vector<int>& a, const vector<int>&
b) {
            return a[1] < b[1];
        });

        int arrows = 1;
        long long arrowPos = points[0][1];

        for (int i = 1; i < points.size(); i++) {
            if (points[i][0] > arrowPos) {
                arrows++;
                arrowPos = points[i][1];
            }
        }

        return arrows;
    }
};
```

20.4 Greedy Proof

Shooting at the earliest ending balloon's end is safe because any valid arrow for that balloon must be no later than its end.

Choosing its end maximizes chance to also hit later balloons.

20.5 Complexity

Time:

$O(n \log n)$

Space:

$O(1)$ extra

21. Full Problem Walkthrough 10 — LeetCode 406: Queue Reconstruction by Height

21.1 Problem Summary

Each person is $[h, k]$.

k means number of people in front with height greater than or equal to h .

Reconstruct the queue.

21.2 How to Identify Greedy

Taller people affect shorter people.

Shorter people do not affect taller people.

Sort by:

height descending

k ascending

Then insert each person at index k .

21.3 C++ Code

```
class Solution {
public:
    vector<vector<int>> reconstructQueue(vector<vector<int>>& people) {
        sort(people.begin(), people.end(), [](const vector<int>& a, const vector<int>&
b) {
            if (a[0] != b[0]) {
                return a[0] > b[0];
            }

            return a[1] < b[1];
        });

        vector<vector<int>> result;
```

```

    for (auto& person : people) {
        result.insert(result.begin() + person[1], person);
    }

    return result;
}
};

```

21.4 Greedy Proof

When inserting a person, all previously inserted people are taller or equal.

Therefore, inserting at index k guarantees exactly k taller-or-equal people before them.

Later shorter people do not affect this count.

21.5 Complexity

Time:

$O(n^2)$

because vector insertion can shift elements.

Space:

$O(n)$

22. Full Problem Walkthrough 11 — LeetCode 135: Candy

22.1 Problem Summary

Children have ratings.

Each child must get at least one candy.

Children with higher rating than adjacent child must get more candy.

Return minimum candies.

22.2 How to Identify Greedy

There are two local constraints:

- left neighbor
- right neighbor

Use two passes.

Left to right handles increasing from left.

Right to left handles increasing from right.

22.3 C++ Code

```

class Solution {
public:
    int candy(vector<int>& ratings) {
        int n = ratings.size();

        vector<int> candies(n, 1);
    }
};

```

```

    for (int i = 1; i < n; i++) {
        if (ratings[i] > ratings[i - 1]) {
            candies[i] = candies[i - 1] + 1;
        }
    }

    for (int i = n - 2; i >= 0; i--) {
        if (ratings[i] > ratings[i + 1]) {
            candies[i] = max(candies[i], candies[i + 1] + 1);
        }
    }

    int total = 0;

    for (int x : candies) {
        total += x;
    }

    return total;
}
};

```

22.4 Greedy Proof

Each pass satisfies one direction of the neighbor constraint.

Taking max in the second pass preserves the first constraint while satisfying the second.

22.5 Complexity

Time:

$O(n)$

Space:

$O(n)$

23. Full Problem Walkthrough 12 — LeetCode 621: Task Scheduler

23.1 Problem Summary

Tasks are represented by capital letters.

Same task must be separated by at least n intervals.

Return minimum intervals needed.

23.2 How to Identify Greedy

The most frequent task creates the main structure.
If max frequency is maxFreq, then there are maxFreq - 1 gaps.
Each gap has length n + 1.
Formula:

$$(\text{maxFreq} - 1) * (n + 1) + \text{numberOfTasksWithMaxFreq}$$

Answer is max of this and total tasks.

23.3 C++ Code

```
class Solution {
public:
    int leastInterval(vector<char>& tasks, int n) {
        vector<int> freq(26, 0);

        for (char task : tasks) {
            freq[task - 'A']++;
        }

        int maxFreq = 0;

        for (int count : freq) {
            maxFreq = max(maxFreq, count);
        }

        int maxCount = 0;

        for (int count : freq) {
            if (count == maxFreq) {
                maxCount++;
            }
        }

        int formula = (maxFreq - 1) * (n + 1) + maxCount;

        return max((int)tasks.size(), formula);
    }
};
```

23.4 Greedy Proof

The most frequent tasks force the minimum number of cooldown gaps.
Other tasks can fill those gaps.
If there are enough other tasks, no idle time is needed, so answer is total task count.

23.5 Complexity

Time:

$O(n)$

Space:

$O(1)$

24. Full Problem Walkthrough 13 — LeetCode 846: Hand of Straights

24.1 Problem Summary

Given hand of cards and group size.

Return true if cards can be rearranged into groups of consecutive cards.

Example:

hand = [1,2,3,6,2,3,4,7,8]

groupSize = 3

Possible groups:

[1,2,3], [2,3,4], [6,7,8]

24.2 How to Identify Greedy

Always start from the smallest remaining card.

If smallest card is x , it must start a group:

$x, x+1, \dots, x+\text{groupSize}-1$

No earlier card can use it.

24.3 C++ Code

```
class Solution {
public:
    bool isNStraightHand(vector<int>& hand, int groupSize) {
        if (hand.size() % groupSize != 0) {
            return false;
        }

        map<int, int> count;

        for (int card : hand) {
            count[card]++;
        }

        while (!count.empty()) {
            int start = count.begin()->first;

            for (int x = start; x < start + groupSize; x++) {
```

```

        if (!count.count(x)) {
            return false;
        }

        count[x]--;

        if (count[x] == 0) {
            count.erase(x);
        }
    }
}

return true;
}
};

```

24.4 Greedy Proof

The smallest remaining card cannot be placed after any smaller card because no smaller card remains.

Therefore, it must begin a consecutive group.

So greedily forming a group from the smallest card is forced.

24.5 Complexity

Time:

$O(n \log n)$

Space:

$O(n)$

25. Full Problem Walkthrough 14 — LeetCode 853: Car Fleet

25.1 Problem Summary

Cars move toward target.

A faster car catches a slower car ahead and becomes part of its fleet.

Return number of fleets reaching target.

25.2 How to Identify Greedy

Sort cars by position descending, closest to target first.

Track arrival time of the slowest fleet ahead.

A car behind forms a new fleet only if its arrival time is greater than the fleet ahead.

25.3 C++ Code

```

class Solution {
public:
    int carFleet(int target, vector<int>& position, vector<int>& speed) {

```

```

vector<pair<int, double>> cars;

for (int i = 0; i < position.size(); i++) {
    double time = (double)(target - position[i]) / speed[i];
    cars.push_back({position[i], time});
}

sort(cars.rbegin(), cars.rend());

int fleets = 0;
double slowestTime = 0.0;

for (auto& [pos, time] : cars) {
    if (time > slowestTime) {
        fleets++;
        slowestTime = time;
    }
}

return fleets;
}
};

```

25.4 Greedy Proof

A car behind can only catch fleets ahead, never cars behind it.

Processing from closest to farthest lets us decide whether each car joins the fleet ahead or forms a new one.

25.5 Complexity

Time:

$O(n \log n)$

Space:

$O(n)$

26. Full Problem Walkthrough 15 — LeetCode 330: Patching Array

26.1 Problem Summary

Given sorted positive array `nums` and integer `n`.

Add minimum numbers so every number in range $[1, n]$ can be formed as a sum of some elements.

26.2 How to Identify Greedy

Maintain smallest missing sum:

miss

Meaning:

we can form every sum in $[1, \text{miss})$

If next number $\text{nums}[i] \leq \text{miss}$, then coverage extends to:

$[1, \text{miss} + \text{nums}[i])$

If $\text{nums}[i] > \text{miss}$, we must patch with miss.

26.3 C++ Code

```
class Solution {
public:
    int minPatches(vector<int>& nums, int n) {
        long long miss = 1;
        int patches = 0;
        int i = 0;

        while (miss <= n) {
            if (i < nums.size() && nums[i] <= miss) {
                miss += nums[i];
                i++;
            } else {
                miss += miss;
                patches++;
            }
        }

        return patches;
    }
};
```

26.4 Greedy Proof

If we can form $[1, \text{miss})$, and the next number is greater than miss, then miss itself cannot be formed.

The smallest and best patch is miss, because it maximally extends coverage to $[1, 2 * \text{miss})$.

26.5 Complexity

Time:

$O(n)$

where n here means $\text{nums.size}()$.

Space:

$O(1)$

27. Common Greedy Edge Cases

27.1 Empty Input

Check whether answer should be 0, false, or empty list.

27.2 One Element

Common in:

- Jump Game
- Candy
- Gas Station
- Boats

27.3 Duplicates

Important for:

- Hand of Straights
- Queue Reconstruction
- Task Scheduler

27.4 Sorting Direction

Wrong sorting direction ruins greedy.

Examples:

- intervals by end time
- queue by height descending
- cars by position descending

27.5 Integer Overflow

Use long long for:

- Patching Array
- large sums
- time calculations

27.6 Boundary Equality

Examples:

- intervals touching may or may not overlap
 - balloon start equal arrow position is burst
 - boat weight equal limit is allowed
-

28. Common Mistakes and Wrong Code

Mistake 1 — Using Greedy Without Proof

If you cannot justify the local choice, the solution may be wrong.

Mistake 2 — Sorting by Wrong Key

For interval removal, sort by end, not start.

Mistake 3 — Confusing Local Maximum With Global Maximum

Greedy needs a safe choice, not just the largest immediate value.

Mistake 4 — Not Handling Equality

Example:

For balloons, new arrow needed only if:

```
points[i][0] > arrowPos
```

not $\geq$.

Mistake 5 — Forgetting Impossible Case

Gas Station needs:

```
total  $\geq$  0
```

otherwise impossible.

Mistake 6 — Not Using long long

Patching Array requires long long for miss.

29. Greedy Interview Explanation Templates

29.1 General Greedy

I make the locally optimal choice because it is safe: there exists an optimal solution that includes this choice. After making it, the remaining problem has the same structure.

29.2 Interval Greedy

I sort by end time because choosing the earliest-ending interval leaves the maximum space for future intervals.

29.3 Jump Game

I track the farthest index reachable so far. If I ever reach an index beyond that, no previous jump could reach it, so the end is impossible.

29.4 Gas Station

If the tank becomes negative at station i , then no station in the current segment can be a valid start. So I reset the start to $i + 1$.

29.5 Patching Array

I maintain the smallest value miss that cannot be formed. If the next number is greater than miss, I must patch miss itself, because it extends coverage the most.

30. Complexity Patterns

Simple greedy scan	$O(n)$	$O(1)$
Sort + greedy	$O(n \log n)$	$O(1)$ or $O(n)$
Interval greedy	$O(n \log n)$	$O(1)$
Heap greedy	$O(n \log n)$	$O(n)$
Counting greedy	$O(n \log n)$ or $O(n)$	$O(n)$

Problem Type	Common Time	Common Space
Greedy with two pointers	$O(n \log n)$	$O(1)$
Greedy stack	$O(n)$	$O(n)$

31. Time-Boxed Practice Guidance

Easy Greedy

Target:

10–20 minutes

Problems:

- Assign Cookies
- Lemonade Change
- Maximum Units on a Truck
- Boats to Save People

Medium Greedy

Target:

25–45 minutes

Problems:

- Jump Game
- Jump Game II
- Gas Station
- Partition Labels
- Non-overlapping Intervals
- Minimum Arrows
- Car Fleet

Hard Greedy

Target:

45–75 minutes

Problems:

- Candy
- Task Scheduler
- Hand of Straights
- Patching Array
- Remove K Digits
- Queue Reconstruction
- Minimum Number of Taps to Open to Water a Garden

32. Expanded Practice Set

32.1 Must-Do Easy

26. Assign Cookies
27. Lemonade Change

- 28. Maximum Units on a Truck
- 29. Boats to Save People
- 30. Best Time to Buy and Sell Stock II
- 31. Can Place Flowers
- 32. Minimum Subsequence in Non-Increasing Order

32.2 Must-Do Medium

- 33. Jump Game
- 34. Jump Game II
- 35. Gas Station
- 36. Partition Labels
- 37. Non-overlapping Intervals
- 38. Minimum Number of Arrows to Burst Balloons
- 39. Queue Reconstruction by Height
- 40. Car Fleet
- 41. Hand of Straights
- 42. Task Scheduler
- 43. Remove K Digits
- 44. Candy

32.3 Strong Interview Practice

- 45. Patching Array
- 46. Minimum Number of Taps to Open to Water a Garden
- 47. Reorganize String
- 48. Distant Barcodes
- 49. Smallest Number From DI String
- 50. Maximum Swap
- 51. Monotone Increasing Digits
- 52. Minimum Deletions to Make Character Frequencies Unique
- 53. Bag of Tokens
- 54. Advantage Shuffle

32.4 Hard / Stretch

- 55. Patching Array
 - 56. Candy
 - 57. Create Maximum Number
 - 58. Minimum Number of Refueling Stops
 - 59. Course Schedule III
 - 60. Remove Duplicate Letters
 - 61. Maximum Performance of a Team
 - 62. Minimum Cost to Hire K Workers
-

33. Suggested Study Order

Step 1 — Basic Local Choice

Start with:

- 63. Assign Cookies
- 64. Lemonade Change
- 65. Boats to Save People

Goal:

Understand smallest sufficient resource and preserving scarce resources.

Step 2 — Reachability Greedy

Solve:

- 66. Jump Game
- 67. Jump Game II
- 68. Gas Station

Goal:

Learn farthest reach and reset logic.

Step 3 — Interval Greedy

Solve:

- 69. Non-overlapping Intervals
- 70. Minimum Number of Arrows
- 71. Meeting Rooms-style interval problems

Goal:

Learn earliest-end greedy.

Step 4 — Partition and Boundary Greedy

Solve:

- 72. Partition Labels
- 73. Car Fleet

Goal:

Learn boundary closure and processing order.

Step 5 — Hard Greedy

Solve:

- 74. Candy
- 75. Task Scheduler
- 76. Hand of Straights
- 77. Patching Array

Goal:

Practice proof-heavy greedy problems.

34. Pattern Decision Table

satisfy maximum children	smallest sufficient resource	Assign Cookies
give change	preserve smaller bills	Lemonade Change
can reach end	farthest reachable index	Jump Game
minimum jumps	BFS-level greedy range	Jump Game II
circular route	reset after bad prefix	Gas Station
split labels	farthest last occurrence	Partition Labels
pair under limit	lightest + heaviest	Boats
remove overlaps	keep earliest ending	Non-overlapping

Problem Clue	Greedy Strategy	Example
		Intervals
burst intervals	shoot at earliest end	Arrows
reconstruct by constraints	sort strong elements first	Queue Reconstruction
neighbor constraints	two passes	Candy
cooldown scheduling	most frequent task formula	Task Scheduler
consecutive groups	start from smallest remaining	Hand of Straights
fleets	process closest to target first	Car Fleet
cover all sums	patch smallest missing	Patching Array

35. Mock Interview — Jump Game II

Interviewer Prompt

Given an array where each element represents maximum jump length, return the minimum number of jumps to reach the last index.

Candidate Clarifying Questions

- Is reaching the last index always possible?
- Can array length be 1?
- Are values non-negative?

Brute Force

Try all possible jumps recursively.

This is exponential.

Greedy Insight

Each jump covers a range of indices.

While scanning the current range, track the farthest index reachable with one more jump.

When we finish current range, we must jump.

Explanation

I maintain:

```
currentEnd = boundary of current jump count
farthest = farthest index reachable from current range
jumps = number of jumps used
```

When $i == \text{currentEnd}$, I have processed all positions reachable with the current number of jumps, so I increment jumps and move currentEnd to farthest.

Final Code

```
class Solution {
public:
    int jump(vector<int>& nums) {
        int jumps = 0;
        int currentEnd = 0;
        int farthest = 0;

        for (int i = 0; i < nums.size() - 1; i++) {
            farthest = max(farthest, i + nums[i]);

            if (i == currentEnd) {
                jumps++;
                currentEnd = farthest;
            }
        }

        return jumps;
    }
};
```

Complexity

Time:

$O(n)$

Space:

$O(1)$

36. Mini Problem 1 — Best Time to Buy and Sell Stock II

Pattern

Greedy take every positive increase.

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int profit = 0;

        for (int i = 1; i < prices.size(); i++) {
```

```

        if (prices[i] > prices[i - 1]) {
            profit += prices[i] - prices[i - 1];
        }
    }

    return profit;
}
};

```

Key idea:

Every increasing segment can be broken into daily gains.

37. Mini Problem 2 — Maximum Units on a Truck

Pattern

Sort by units per box descending.

```

class Solution {
public:
    int maximumUnits(vector<vector<int>>& boxTypes, int truckSize) {
        sort(boxTypes.begin(), boxTypes.end(), [](const vector<int>& a, const
vector<int>& b) {
            return a[1] > b[1];
        });

        int total = 0;

        for (auto& box : boxTypes) {
            int take = min(box[0], truckSize);

            total += take * box[1];
            truckSize -= take;

            if (truckSize == 0) {
                break;
            }
        }

        return total;
    }
};

```

38. Mini Problem 3 — Monotone Increasing Digits

Pattern

If a digit violates monotonic order, reduce previous digit and make suffix 9.

```
class Solution {
public:
    int monotoneIncreasingDigits(int n) {
        string s = to_string(n);
        int marker = s.size();

        for (int i = s.size() - 1; i > 0; i--) {
            if (s[i - 1] > s[i]) {
                s[i - 1]--;
                marker = i;
            }
        }

        for (int i = marker; i < s.size(); i++) {
            s[i] = '9';
        }

        return stoi(s);
    }
};
```

39. Mini Problem 4 — Minimum Deletions to Make Character Frequencies Unique

Pattern

Greedily reduce duplicate frequencies.

```
class Solution {
public:
    int minDeletions(string s) {
        vector<int> freq(26, 0);

        for (char c : s) {
```

```

        freq[c - 'a']++;
    }

    unordered_set<int> used;
    int deletions = 0;

    for (int f : freq) {
        while (f > 0 && used.count(f)) {
            f--;
            deletions++;
        }

        if (f > 0) {
            used.insert(f);
        }
    }

    return deletions;
}
};

```

40. Common Edge Cases Checklist

Before submitting greedy code, test:

- empty input
 - one element
 - all equal values
 - already optimal order
 - reverse order
 - impossible case
 - boundary equality
 - duplicate values
 - very large values
 - sorting tie cases
 - first/last interval touching
 - negative or zero values if allowed
-

41. What to Master Before Chapter 32

Before moving to Interval Problems, make sure you can:

- Explain greedy local choice
- Prove greedy with exchange argument
- Prove greedy with staying-ahead argument

- Solve Assign Cookies
 - Solve Jump Game
 - Solve Jump Game II
 - Solve Gas Station
 - Solve Partition Labels
 - Solve Boats to Save People
 - Solve Non-overlapping Intervals
 - Solve Minimum Arrows
 - Solve Candy
 - Solve Task Scheduler
 - Solve Hand of Straights
 - Explain why some problems need DP instead of greedy
-

42. Final Cheat Sheet

Sort + Assign

```
sort(a.begin(), a.end());
sort(b.begin(), b.end());
```

Use smallest sufficient resource.

Farthest Reach

```
farthest = max(farthest, i + nums[i]);
```

Interval Earliest End

```
sort(intervals.begin(), intervals.end(), [](auto& a, auto& b) {
    return a[1] < b[1];
});
```

Two-Pointer Pairing

```
sort(nums.begin(), nums.end());

int left = 0;
int right = nums.size() - 1;

while (left <= right) {
    if (nums[left] + nums[right] <= limit) {
        left++;
    }

    right--;
}
```

Coverage Greedy

```
long long miss = 1;
```

```
while (miss <= n) {  
    if (i < nums.size() && nums[i] <= miss) {  
        miss += nums[i++];  
    } else {  
        miss += miss;  
        patches++;  
    }  
}
```

43. Core Takeaway

Greedy is about making a local choice that is globally safe.

The code is often short.

The proof is the hard part.

Use greedy when you can clearly justify:

This choice never makes the optimal answer worse.

The most common greedy patterns are:

- sort and assign
- choose earliest ending interval
- track farthest reach
- pair smallest with largest
- preserve scarce resources
- close boundary when forced
- patch smallest missing coverage

Greedy becomes easier when you ask:

What choice is forced?

What choice leaves the most flexibility?

What choice keeps me farthest ahead?

If you can answer those questions, you are probably close to the correct greedy solution.